

Actividad: Análisis de código y refactorización

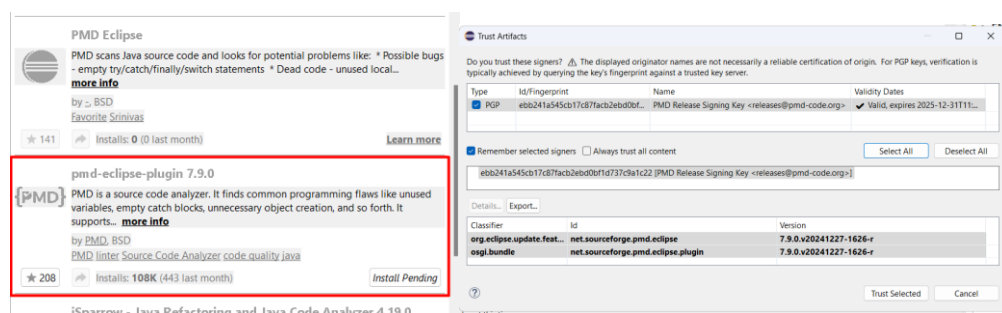
Realiza en pareja las siguientes acciones de análisis de código y refactorización.

1 Analizador de código en Eclipse (PMD)

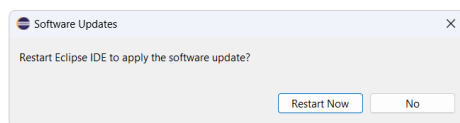
Crea un proyecto Eclipse con los paquetes de java adjuntos a la actividad.

Instala el siguiente plugin/extensión de eclipse que servirá como analizador de código, para la aplicación de reglas de buenas prácticas, code style, etc.

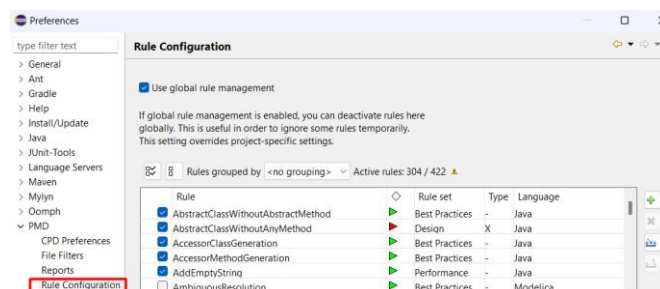
Instala ambas extensiones:



<https://marketplace.eclipse.org/content/pmd-eclipse-plugin>



Localiza donde se configuran las reglas PMD sobre buenas prácticas, diseño, etc de los diferentes lenguajes.



1.1 Configuración de Reglas

A continuación completa la tabla con las reglas que consideres importantes en el lenguaje **Java** clasificada por tipo de regla (Buenas practicas, diseño...) y nivel de prioridad (Bloqueante, critica...)

Métrica	Tipo	Prioridad	Descripción en español
AddEmptyString			
MethodNamingConventions			
AbstractClassWithoutAbstractMethod			
StringInstantiation			
ForLoopVariableCount			
PackageCase			
StringToString			
SystemPrintln			
ShortMethodName			
DoNotThrowExceptionInFinally			
OneDeclarationPerLine			

Añade a continuación un ejemplo de cada tipo de regla Java que no haya aparecido anteriormente.

Métrica	Tipo	Prioridad	Descripción en español

1.2 Analiza el código

Carga en un proyecto eclipse los paquetes Java adjuntos a la actividad y analiza el código. Adjunta los resultados.

1.3 Modifica el código para provocar los siguientes “problemas”

Adjunta los resultados:

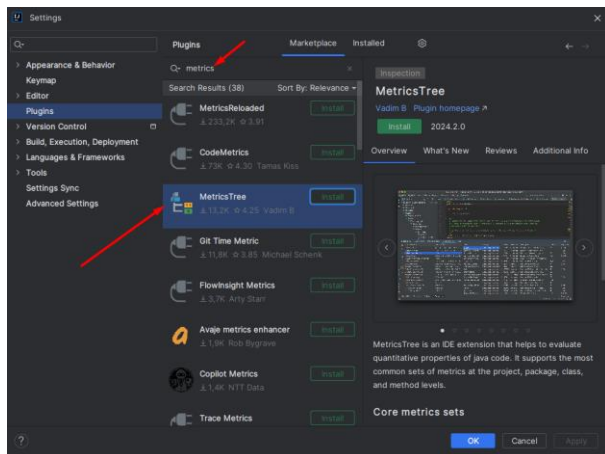
1. MethodNamingConventions
2. AbstractClassWithoutAbstractMethod
3. StringInstantiation
4. StringToString
5. ShortMethodName

1.4 Realiza un análisis de Código duplicado

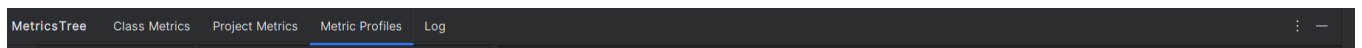
Adjunta los resultados de un análisis de código duplicado (**CPD Preferences**).

2 Analizador de código en IntelliJ (MetricsTree)

En **Overview** de esta extensión tienes las características de lo que esta extensión permite analizar de tu código. Como su nombre indica está destinado a calcular métricas de calidad de código.



Tras la instalación se incorporan varias vistas de **MetricsTree**, a nivel de análisis de clase, métodos y proyecto.



2.1 Analiza el código

Carga en un proyecto Java IntelliJ los paquetes Java adjuntos a la actividad y analiza el código.

Adjunta los resultados a nivel de proyecto **Project Metrics** utilizando el grafico de barras



2.2 Métricas de calidad del código

Averigua el significado de las siguientes métricas y adjunta una captura de un ejemplo

Métrica	Significado	Descripción en español	Captura de un ejemplo del análisis de la métrica sobre una clase
LOC			
WMC			
CBO			
RFC			
LCOM			
DIT			
CC			

2.3 Configuración de métricas

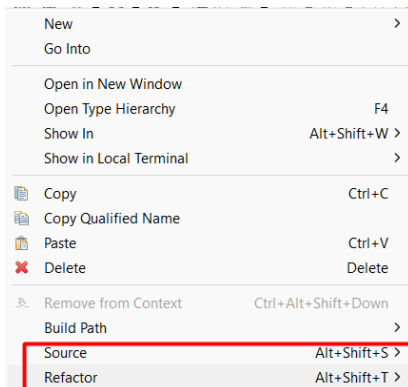
Desde la vista **Metrics Profiles**.

1. ¿Qué valor de Acoplamiento entre objetos determina un Alto acoplamiento (dependencia entre clases)?
2. ¿Qué valor de complejidad ciclomática y líneas de código determinan que existe demasiada lógica en una clase (Brain Class)?
3. ¿Y en un método (Brain Method)?
4. ¿Cuántos métodos son considerados demasiados?
5. ¿Cuántos parámetros son considerados demasiados?

3 Refactorización del paquete adjunto “refactoriza”

Adjunto a la actividad se encuentra un paquete **refactoriza** que tiene una clase FARMACIA.java que necesita una refactorización urgente:

Aplica refactorizaciones utilizando las opciones de Eclipse



Documenta con capturas como realizas las siguientes refactorizaciones.

1. Formatea estructura de código aplicando convenciones y buenas prácticas Java
2. Extrae herencia de los medicamentos (superclase y clases hijas).
3. Renombra métodos a minúsculas y nombres de clase con 1ª letra mayúscula, el resto minúscula y sin guiones.
4. Elimina código duplicado.
5. Extrae constantes para tipos de medicamentos usando mayúsculas para nombrar las constantes.
6. Utiliza nombres de paquetes descriptivos dividiendo responsabilidades tomando como base: xyz.endes.farmacia
7. Mueve el main a una clase Principal desde la que inicializar el programa.
8. Otra refactorización que creas necesaria.
9. Otra refactorización que creas necesaria.
10. Otra refactorización que creas necesaria.

4 Entrega

Documento PDF con el nombre y apellidos de la pareja y los resultados obtenidos.